

SurveyArc Technical Audit and Valuation Report

Prepared as an independent third-party style technical assessment

Date: May 28, 2026

Primary commercial focus: Survey creation, survey programming, survey management, response collection, quota, rules, campaign readiness, and internal handoff value

Additional platform scope reviewed: ticket management, SLA management, RBAC, user management, organization/project management, contact/audience management, Salesforce, pricing/payment foundations, marketplace/integrations/webhooks, audit/archive/metrics, mail relay, key server, Redis-backed caching, and frontend/backend API layers

Executive Opinion

SurveyArc is not a small demo application. The repository contains a broad, multi-service product foundation with survey authoring as the central business value, plus supporting modules for organization management, project management, contacts, campaigns, quota, translations, RBAC, ticketing, SLA operations, Salesforce, payments/pricing, marketplace, integrations, webhooks, mail relay, and encryption-related services.

The survey product is the main asset being valued in this report. It already includes a visual survey builder, a large question-type library, blocks and ordering, survey runtime rendering, response handling, quota/rule infrastructure, translation support, templates, theme support, campaign integration points, and an XML survey programming mode. These features represent substantial development effort and give an acquiring internal team a meaningful head start compared with starting from zero.

Recommended Price Position

Recommended opening pitch for complete code transfer: **INR 18,00,000 to INR 22,00,000**

Practical negotiation target: **INR 14,00,000 to INR 17,00,000**

Minimum sensible floor for the full repository with KT: **INR 11,00,000 to INR 12,50,000**

The above assumes an as-is code transfer, documentation bundle, and reasonable knowledge transfer support. It is not priced as a fully supported commercial SaaS.

This assessment uses an internal-use acquisition lens. Since the buyer intends to use and extend the tool internally, the valuation should not be judged only by public SaaS readiness. Internal-use buyers can accept unfinished polish if the architecture, data model, and business modules save significant development time.

1. Scope and Evidence Reviewed

The repository was reviewed across frontend, backend, service, model, route, schema, and integration layers. The most important observation is that the survey module is not isolated: it is connected to projects, organizations, users, contacts, campaigns, quotas, translations, themes, responses, and permission surfaces. The ticket/SLA system is also a significant secondary product line and should be mentioned in the sale story as additional platform value.

Evidence Area	Observed Scope	Business Meaning
Source size	Approximately 967 Python/JavaScript/React source files reviewed by inventory scan.	This is a multi-module platform, not a few screens or a prototype script.
Backend API surface	Approximately 501 FastAPI route decorators across backend modules.	Large API surface covering surveys, questions, projects, users, RBAC, campaigns, contacts, tickets, SLA, and support operations.
Frontend API proxy layer	Approximately 252 Next.js API proxy route files under the Postgres API route structure.	Frontend is already wired to backend-style service boundaries, reducing handoff friction.
Question configuration library	More than 40 dedicated question configuration components plus shared question editor and runtime forms.	Survey builder has broad market-research/questionnaire coverage.
Ticket/support UI	Ticket dashboard, ticket detail, agent workflow components, SLA admin, support groups, support teams, RBAC management.	Secondary operational platform exists and increases overall asset value.
Additional business modules	Pricing/payment/order/invoice foundations, marketplace, integrations, webhooks, domains, audit events/logs, archive, metrics, tags, Salesforce sync, export utilities, ownership transfer, email/WhatsApp UI surfaces.	These are additional product and handoff assets that were identified in the repo-wide rescan.
Build/compile status	Frontend build completed with warnings; backend compile check completed successfully in earlier verification.	The project is not fully hardened, but it is structurally executable and developable.
Testing status	No meaningful automated test suite was identified during this audit pass.	Buyer should budget stabilization and regression testing after acquisition.

2. Product Summary: What Has Been Built

The repository can be described as a survey-led internal research and operations platform. The strongest and most relevant area for the buyer is the survey product. However, the surrounding modules matter because they make the survey product usable in an organization: users can be managed, projects can hold surveys, contacts can be imported, campaigns can send survey links, quotas can control respondent segments, and operational tickets/SLA workflows can support internal service management. A second full scan also identified commerce, subscription, integration, audit, Salesforce, export, WhatsApp, and ownership-transfer foundations that should be treated as additional included value, even if they are not the primary reason for acquisition.

Product Area	Status	What Is Done	What Still Needs Work
Survey builder and survey management	Major foundation done	Survey CRUD, project-level survey listing, builder navigation, question creation/editing/deletion, ordering, blocks, page breaks, templates, XML mode, preview, runtime rendering, response collection.	UX polishing, validation consistency, autosave/save-state clarity, stronger tests, schema normalization, production-grade QA.
Question library	Broad coverage done	Many research-grade question types including NPS, OSAT, matrix, ranking, MaxDiff, conjoint, semantic diff, price sensitivity, Gabor-Granger, monadic/sequential/forced exposure, contact fields, file upload, media, and screens.	Each advanced type needs respondent-side QA, analytics validation, export validation, and edge-case testing.
Survey runtime and responses	Functional but needs hardening	Survey form rendering, block progression, answer UI, response APIs, response overview pages, survey demo/preview areas.	End-to-end respondent journey should be tested across every question type and device size.
Rules, quota, translations	Important foundation present	Question logic fields, rules API, quota API with evaluate/increment, translation structures, translation CSV import/export/validation.	Needs stronger rule builder UX, logic engine completeness, quota reporting, and test coverage.
Contacts, audience, campaigns	Integration layer present	Contacts, lists, contact emails/phones/socials, audience files, campaign creation/scheduling, campaign result models, mail relay integration.	Provider integrations for SMS/WhatsApp/voice are placeholders; email relay and tracking need production configuration.
Project, organization, user, RBAC	Strong internal foundation	Organization/project CRUD, user management, roles, permissions, grants, denials, project members, favorites, milestones, attachments, cache invalidation.	Permission model needs final review, caching strategy cleanup, and clear admin documentation.
Ticket management and SLA	Large secondary module	Ticket CRUD, comments, attachments, collaborators, worklogs, assignments, SLA timers, pause/resume, support groups, teams, routing, categories, taxonomies, templates.	Should be treated as a second product line needing QA, migrations, and process documentation.

Infrastructure and services	Useful but needs ops polish	Docker-oriented structure, Redis services, encryption middleware, key server, mail relay, background processors.	Production secrets, environment docs, monitoring, migration discipline, and deployment runbooks are required.
Commerce, integrations, audit, Salesforce, exports	Additional foundation present	Pricing plan routes, order/payment/invoice models, Razorpay frontend utilities, marketplace and integration models/routes, webhooks, domains, audit logs/events, archive, metrics, Salesforce contact/account/sync/campaign routes, Excel/SPSS export utilities, analytics helpers.	These modules need business-rule validation, security review, and documentation before being positioned as production-ready.

3. Survey System Deep Dive

The survey system is the main value driver. It combines a visual builder, backend schema, frontend state management, question configuration forms, survey runtime, response capture, translation support, quota/rule foundations, template creation, and campaign-related delivery surfaces. For an internal buyer, this survey foundation is the strongest reason to acquire the repository instead of building from scratch.

3.1 Survey Data Model and Backend APIs

The backend survey model stores the survey as an organization/project-owned object with survey identity, name, slug, version, status, settings, question order, theme association, blocks, and block order. This is important because it supports both simple linear surveys and more complex block-based survey programming.

Survey Capability	Evidence in Codebase	Assessment
Survey CRUD	Backend route supports create, get by survey ID, list, project-level list, update/patch, and delete.	Core survey lifecycle exists and can be extended by the buyer's engineering team.
Survey status	Survey schema/model carries status fields; frontend displays draft/live states.	Useful foundation, but status synchronization between inner builder and outside listing needs final consistency checks.
Question order	Survey payload supports <code>question_order</code> and frontend maps <code>questionOrder</code> to backend shape.	Necessary for stable survey rendering and survey programming.
Blocks and block order	Survey stores <code>blocks</code> and <code>block_order</code> ; frontend builder supports selected blocks and per-block question order.	Important feature because complex surveys usually need sections, page breaks, randomized blocks, and grouping.
Responses count and response listing	Survey routes expose response count and response listing per survey.	Basic reporting and operational visibility are present.
Dummy response generation	Survey routes include generate-dummy, task status, cancel, and list dummy-generation tasks.	Useful for demo/testing, but generated dummy artifacts should be controlled and excluded from commits.
Locales	Survey route exposes survey locales and question translation APIs exist.	Translation architecture is present and valuable for multilingual internal survey work.

3.2 Question Data Model

Questions are modeled with both internal identity and human-facing labels. The backend stores `question_id` as the primary key and `serial_label` as the reference label used for programming, logic, exports, and human readable references such as Q1, Q2, Q3, etc. This is the correct direction because internal IDs should not be exposed as the main survey-programming reference.

Question Field	Purpose	Value
<code>question_id</code>	System-level unique identifier, primary key.	Prevents internal collisions and allows stable database references.
<code>serial_label</code>	Human-facing survey programming label such as Q1, Q2, S1, etc.	Better UX for logic, exports, XML mode, and analyst workflows.
<code>type</code>	Controls renderer/config editor behavior.	Supports broad question library and screen types.
<code>label</code>	The actual respondent-facing question text.	Core survey content field.
<code>description</code>	Helper text or subtext.	Supports richer survey authoring.
<code>config</code>	Question-type-specific JSON configuration.	Enables flexible advanced question types without changing the base table repeatedly.
<code>logic</code>	Per-question logic and branching actions.	Important for future survey programming maturity.
<code>translations</code>	Localized labels/descriptions/configs.	Important for multilingual internal deployments.

3.3 Question Backend APIs

The question backend is significantly more complete than a basic CRUD module. It includes list questions for a survey, create, bulk create, get, update, delete, bulk fetch, translated question retrieval, translation structures, locale initialization, resync, CSV upload, CSV export, and CSV validation.

Question API Area	What Exists	Why It Matters
Question CRUD	Create, read, update, delete, and survey-level list APIs.	Minimum foundation for a real builder.
Bulk create	<code>/bulk-create</code> endpoint and frontend model support.	Important performance improvement for survey templates and XML compiler output.
Bulk fetch	Bulk question retrieval by IDs.	Useful for runtime rendering and builder hydration.
Translations	Question translation coverage, blank translation structure, update all translations, update locale, initialize locale, delete locale, delete all.	Substantial enterprise/internal-use feature, especially for survey operations across languages.
CSV workflows	Translation CSV upload, export, and validation.	Matches how research teams often manage translations outside the app.
Logic schema	Logic condition/action/rule schemas reference serial labels or question IDs.	Provides the base for survey branching and rule authoring.

3.4 Question Type Library

The frontend question library is one of the most valuable parts of the survey system. The codebase includes a large set of question types commonly needed by market research, CX, internal feedback, and panel-style survey operations.

Question Category	Implemented/Represented Types	Business Value
Basic choice	Multiple choice, dropdown, checkbox, yes/no, picture choice, legal/consent.	Covers standard questionnaire needs.
Rating and CX	NPS, OSAT, rating, opinion scale, smiley rating, image click rating.	Supports customer satisfaction, feedback, and experience research.
Text and data capture	Short text, long text, number, date, contact email, phone, address, website.	Supports respondent profile, feedback, and operational data collection.
Media and files	Video, file upload, Google Drive, Calendly.	Useful for richer respondent interactions and appointment/file flows.
Matrix and grid	Matrix, table grid, multi grid, matrix rating, side-by-side, comparison grid.	Important for research surveys where multiple attributes/products are evaluated.
Research/advanced	MaxDiff, conjoint, semantic differential, Gabor-Granger, price sensitivity, Bayes acquisition, weighted multi.	These are high-value research features that are expensive to design and test from scratch.
Stimulus testing	Monadic test, sequential monadic, forced exposure, image click rating.	Relevant for product, ad, packaging, and concept testing.
Segmentation	Segmentation selector, persona quiz, Likert, ranking.	Supports respondent grouping and structured insight workflows.
Flow and screens	Welcome screen, end screen, redirect URL/page-break style structure through builder blocks.	Required for professional survey flows and respondent experience.

Dedicated configuration components observed include Address, AutoSum, BayesAcq, Calendly, ComparisonGrid, Conjoint, Date, Email, EndScreen, FileUpload, ForcedExposure, GaborGranger, GoogleDrive, ImageClickRating, Legal, Likert, Matrix, MatrixRating, MaxDiff, Monadic, MultiGrid, Number, OSAT, Options, PersonaQuiz, PhoneNumber, PictureChoice, PriceSensitivity, Ranking, Rating, Redirect, SegmentationSelector, SemanticDiff, SequentialMonadic, SideBySide, Slider, SmileyRating, Statement, Submit, TableGrid, Text, Turf, Video, Website, WeightedMulti, Welcome, and YesNo configuration components.

3.5 Survey Builder Frontend

The survey builder is not merely a form submission page. It includes the question maker page, question editor panel, question list, tabs, survey toolbar, flow/quota/rule/theme/translation sections, drag/drop ordering concepts, block handling, selected block state, optimistic question creation, and backend refresh after saves.

Builder Area	Implemented Behavior	Assessment
Question creation	User can create questions with label, serial label, type, config, description, and required flags.	Core authoring path exists; validation warnings for missing names/labels are being improved.
Serial label UX	Serial labels are visible and editable while internal question IDs stay behind the scenes.	Correct for analyst-friendly workflows and future XML/logic references.
Question editing	Editor panel supports editing labels, serial labels, descriptions, config forms, and save/update behavior.	Functional and important, but needs final UX polish around save state and validation messages.
Blocks	Builder maintains block order and per-block question order.	Important for page breaks, sections, and future randomization.
Optimistic updates	Question creation uses temporary IDs and later replaces them with backend IDs.	Improves perceived speed, but must be carefully tested to avoid order mismatch.
Bulk/template creation	Template creation uses bulk question creation and then updates survey order.	Good improvement over one API call per question and useful for imported/XML surveys.
Survey status display	Builder and outer survey list both display status but have shown draft/live mismatch risk.	Needs final status-source-of-truth cleanup before handoff.

3.6 XML Survey Programming Mode

A major recent addition is XML survey programming mode. This is valuable because experienced survey programmers often prefer a text/programming style for faster survey creation, especially when building many similar surveys or importing from structured specs.

XML Capability	Observed Behavior	Value
Toggle entry point	XML mode can be toggled from the survey question section.	Allows normal visual-builder users and technical survey programmers to use the same screen.
XML editor	Editor accepts survey XML and displays compiled preview.	Gives programmers a faster authoring path.
Serialization	Survey blocks/questions can be serialized back to XML.	Supports round-tripping and hand-editing existing surveys.
Compilation preview	Compiled preview shows blocks, question count, labels, types, options, and serial labels.	Reduces risk before saving compiled survey output.
Persistence	XML output can be saved back through question and survey APIs, including deletion of stale questions by serial reference.	Strong foundation for a Fosta/Decipher-style programming workflow.
Remaining work	Formal XML grammar, validation messages, import/export docs, advanced logic syntax, and tests are still needed.	This should be treated as a high-potential feature, not yet a fully mature compiler product.

3.7 Survey Runtime and Response Collection

The frontend includes runtime components for rendering surveys and capturing answers. There are answer UI components, survey form screens, demo survey page, response overview, and response models/providers. This means the product has moved beyond authoring and into actual respondent execution.

Runtime Area	Done	Remaining
Survey render flow	Survey form components render questions and support block/page progression.	Needs complete QA across all question types and edge cases.
Answer capture	Answer UI and response APIs are present.	Needs validation for required fields, partial responses, resume behavior, and analytics/export consistency.
Survey preview/demo	Demo survey page and preview-related flows exist.	Should be stabilized as a formal preview/test mode for survey programmers.
Responses dashboard	Response overview pages and response model calls are present.	Analytics depth and exports should be expanded for client operations.

3.8 Rules, Logic, and Quota

Survey logic and quota are key requirements for professional survey operations. The repository already includes a rules route, rule models, quota models, quota APIs, quota frontend components, and question-level logic fields. This is a meaningful foundation even if additional business testing is required.

Area	Current Capability	Assessment
Rules API	Rule create, list, get, update, and delete by survey/question context.	Backend foundation for branching and skip/display logic exists.
Question logic schema	Logic condition/action/rule structures use question references and operators.	Good base for a visual logic builder and XML logic compilation.
Quota backend	Create quota, get quota, list by survey, evaluate quota, increment quota, update quota, delete quota.	Core quota lifecycle exists and is valuable for respondent balancing.
Quota models	SurveyQuota, SurveyQuotaCell, and SurveyQuotaEvent style modeling.	Indicates quota is modeled as more than a flat number.
Quota frontend	Quota tab, quota create form, and question quota modal patterns are present.	Good internal foundation; needs polish and analytics reporting.
Remaining work	End-to-end testing, cross-quota logic, respondent termination/over-quota pages, reporting, and XML syntax support.	Should be a focused hardening area before heavy production usage.

3.9 Translation and Localization

Multilingual support is present at the question level and is supported by translation coverage, initialization, blank translation structure, update APIs, CSV import, CSV export, and CSV validation. For survey work, this is a high-value feature because translation often happens in spreadsheets and external workflows.

Translation Feature	Current State	Value
Translation storage	Question translations are stored as JSON with label, description, and config-level support.	Flexible enough for varied question types.
Coverage checks	Survey-level translation coverage route exists.	Helps operators identify incomplete translations.
CSV workflows	Upload, export, and validate CSV routes exist.	Practical for real survey operations.
Frontend translation page/components	Translation page/grid/model/provider components exist.	Shows translation workflow is intended as a product feature, not only backend storage.

3.10 Themes, Preview, and Survey Presentation

Theme models, theme routes, theme provider/model files, and frontend theme pages/components are present. There are also survey demo and survey preview oriented screens. This gives the buyer a base for branded internal surveys.

Presentation Area	Done	Needed
Themes	Theme CRUD/toggle/duplicate style backend support and frontend theme page.	Design QA, tenant defaults, preview consistency, and documented theme schema.
Demo survey	Demo page and survey runtime page exist.	Standardized preview mode and respondent-mode distinction.
Welcome/end/redirect screens	Question/screen types exist and have config components.	Final UX testing across builder, runtime, and XML.

3.11 Survey-Related Campaign, Contact, and Audience Support

The survey product is also connected to contact and campaign modules. This matters because a survey tool is not only a builder; it also needs a way to distribute survey links and track respondent/channel outcomes.

Distribution Area	What Exists	Assessment
Contacts	Contacts, contact emails, phone numbers, social channels, contact lists, list members, CSV/import-style flows.	Solid data foundation for survey audience management.
Audience files	Audience file upload/create/list/update/delete routes and frontend manager component.	Useful for research operations and campaign-based distribution.
Campaigns	Campaign CRUD, scheduler, sender service, outbox processor, campaign results, analytics routes, email/SMS/WhatsApp/voice channel fields.	Very useful extension to survey product; email is more concrete than SMS/WhatsApp/voice provider integrations.
Mail relay	Separate Node/Express relay supports generic send and kind-based templates for campaign email, SLA, ticket, and calendar messages.	Good architectural separation; needs production SMTP/provider configuration and monitoring.
Participant sources	Participant source CRUD, exit pages, error messages, stats, generate URL, click/start/complete tracking.	Important for panel/source tracking and survey operations.

4. Ticket Management and SLA System

Although the buyer may primarily want SurveyArc for surveys, the repository also contains a significant ticket management and SLA system. This should be mentioned in the commercial story as additional platform value. It can be positioned as an internal operations/helpdesk module that the buyer may use immediately or later.

4.1 Ticket Backend

The ticket backend is broad. It includes ticket creation, list, get, update, delete, root cause setting, counts, comments, attachments, collaborators, group/team/agent assignment, participants, worklogs, SLA pause/resume, and SLA timers. The ticket model includes statuses, priority, severity, platform, requester details, feature/impact/root-cause taxonomies, assignment fields, category and product fields, SLA fields, timing fields, custom fields, metadata, follow-up data, and relationships with comments, tags, attachments, watchers, events, collaborators, worklogs, and SLA status.

Ticket Feature	Status	Business Meaning
Ticket lifecycle	Done foundation	New/open/pending/on-hold/resolved/closed/canceled style lifecycle exists.
Assignments	Done foundation	Tickets can be associated with assignee, group, team, and agent.
Comments	Done foundation	Public/internal comment workflows and notification hooks exist.
Attachments	Partial	Attachment models/routes exist, but storage/security policy should be production reviewed.
Collaborators and participants	Done foundation	Supports multi-person operational resolution workflows.
Worklogs	Done foundation	Useful for time tracking and internal accountability.
Ticket categories and taxonomies	Good foundation	Categories, subcategories, products, features, impact, and root causes are modeled.
Notifications	Integrated	Mail relay supports ticket comment, SLA assigned, warning, and breach email templates.

4.2 SLA System

The SLA module is unusually broad for an internal tool. It includes SLA CRUD, bulk operations, activate/deactivate, duplicate, publish/archive, match/effective SLA lookup, stats, compliance, validation, simulation, export/import, versioning, dependency checks, cleanup, objectives, credit rules, and ticket-level SLA timers/pause/resume.

SLA Feature	Implemented Scope	Value
SLA configuration	SLA model stores target matrix, pause rules, reminder policy, escalation policy, KPI targets, exclusions, penalties, aggregation, scope, precedence, lifecycle, versioning, effective dates.	Strong enterprise-style SLA foundation.
Objectives	SLA objectives have their own model and API lifecycle.	Allows multiple SLA dimensions such as response/resolution style targets.
Credit rules	Credit rule model and APIs exist.	Supports contractual/compliance-style penalty or credit behavior.
Simulation and validation	Validate and simulate endpoints exist.	Useful for admin confidence before publishing SLA changes.
Compliance and stats	Stats and compliance routes exist.	Basis for SLA reporting dashboards.
Frontend admin	SLA admin panel, dialog form, objective form, credit rule form, select, table.	Large amount of frontend work already completed.

4.3 Support Groups, Teams, Routing, Categories, Templates

Module	What Is Done	Remaining Work
Support groups	Group CRUD and group member CRUD with roles/proficiency concepts.	User-stub implementation should be cleaned and documented.
Support teams	Team CRUD, calendar assignment/removal/get, member CRUD, stats.	Calendar and capacity logic need full operational QA.
Routing policies	Routing policy CRUD and evaluate route.	Routing behavior needs business-rule signoff.
Business calendars	Calendar CRUD, hours and holidays support.	Timezone, holiday, and SLA interaction testing required.
Ticket templates	Template CRUD, toggle, duplicate, regenerate API key, create-ticket from template, usage logs, stats.	Security and API-key usage policy should be documented.
Ticket frontend	Ticket dashboard, ticket list, detail screen, raise ticket form, filters, assignee/group/team/tag selectors, agent panels, worklog/comment components.	Needs UX pass, role-based visibility checks, and end-to-end testing.

5. Organization, Project, User Management, and RBAC

The platform includes a meaningful internal administration foundation. This matters for SurveyArc because survey operations need organizations, users, roles, projects, access control, member management, and project-based survey ownership.

5.1 User and Organization Management

Area	Current Capability	Assessment
Users	Create, get, update, soft delete/suspend/activate, track login, list by org, session get/delete, get by email, admin create, batch fetch.	Good internal user-management base.
Organizations	Create, update, get, soft delete, list by owner, settings get/update, members, activate, suspend.	Supports multi-organization ownership and admin controls.
Redis caching	Org/user/session related Redis cache helpers and invalidation logic exist.	Useful for performance, but serialization errors and cache consistency should be cleaned.

5.2 Project Management

Project management is important because surveys are nested under projects. The project route is large and includes creation, listing, get/update/delete, members, bulk members, access checks, user accessible projects, progress recompute, milestones, tags, attachments, survey patching, status management, search, bulk operations, favorites, recent activity, stats, and cache refresh.

Project Feature	Status	Why It Matters
Project CRUD and listing	Implemented	Required foundation for organizing surveys internally.
Project members and access	Implemented with RBAC assignment helper logic.	Supports team collaboration and access scoping.
Milestones, tags, attachments	Implemented foundation.	Adds project-management maturity beyond basic folder structure.
Project status and progress	Status set/allowed and progress recompute features exist.	Helps internal tracking of survey/project delivery.
Favorites and recent activity	Implemented.	Improves internal user productivity.

5.3 RBAC and Permissions

RBAC is present across backend models/services/routes and frontend screens. The repository includes roles, permissions, role-permission mappings, user role assignments, custom grants, explicit denials, effective permission resolution, permission management services, and frontend RBAC management tabs.

RBAC Area	What Is Done	Remaining Work
Roles	Role assignment service, role assignment routes, frontend role management.	Seed data and role naming should be documented clearly.
Permissions	Permission CRUD, module grouping, role-permission add/remove/list, permission checking.	Permission matrix should be finalized for survey-only buyer workflows.
Custom grants	User custom permission grant/revoke/bulk grant services exist.	Admin UX and audit trail need final hardening.
Explicit denials	Permission deny model/service logic exists.	Conflict resolution behavior should be documented and tested.
Frontend RBAC	RBACManagement, RolesTab, PermissionsTab, MappingsTab, UserAccessManager, ProtectedAction, RoleBadge components.	Needs final QA for sidebar reload/performance and permission caching behavior.

6. Mail Relay, Key Server, Redis, and Infrastructure

The repository includes supporting services that are relevant for an internal handoff. They are not fully production-polished, but they demonstrate that the architecture already considered messaging, key management, encrypted payload handling, background jobs, and caching.

Infrastructure Area	Current State	Assessment
Mail relay	Separate Node/Express service with health endpoint, bearer-token auth, generic email send, kind-based templates for SLA, ticket, calendar, and campaign email.	Good service separation. SMS/WhatsApp/voice are currently placeholders and should be integrated with real providers if needed.
Campaign outbox	Backend outbox processor and scheduler concepts exist; relay health check controls batch sending.	Correct architectural direction, but production monitoring/retry policy is needed.
Key server	FastAPI key server returns encrypted AES key material by key ID.	Useful encryption architecture, but secrets/key rotation and deployment docs are required.
Decrypt middleware	Middleware handles encrypted payloads and passes through unencrypted/multipart requests.	Useful for secure client/server flows, but should be reviewed before production exposure.
Encrypt GET middleware	GET JSON response encryption hooks exist and call remote key service.	Hardcoded remote URL and fallback behavior should be cleaned for handoff.
Redis services	Redis wrappers exist for surveys, questions, projects, contacts, campaigns, tickets, SLA, support, themes, responses, RBAC, rules, and taxonomies.	Performance-oriented work is present, but cache invalidation and serialization require QA.
Docker/local orchestration	Docker-related structure exists for local multi-service operation.	Needs clean setup docs, environment variables list, and deployment runbook.

6A. Additional Modules Found in Full Repository Scan

A second repo-wide scan identified additional modules that were not emphasized enough in the earlier report version. These should not replace the survey-first valuation story, but they should be mentioned as included platform assets because they add future optional value for the buyer's internal team.

Additional Area	Files/Capabilities Observed	Commercial Meaning	Readiness Note
Pricing, orders, payments, invoices	Pricing plan APIs, payment/order/invoice models and routes, frontend PricingPage, create-order and verify-payment API routes, Razorpay utilities and script loader.	Provides a foundation for subscriptions, billing records, plan display, payment verification, and future monetization.	Useful foundation, but payment flows need production payment-gateway QA, invoice rules, tax/GST validation, and secrets cleanup.
Marketplace and integrations	Marketplace model/routes, integration model/routes, marketplaceModel, integrationModel, provider templates, email provider configuration surfaces.	Supports the idea of installable/configurable integrations per organization.	Currently best described as a foundation. Actual third-party app lifecycle and permission review need more work.
Webhooks and domains	Webhook model/routes with active state, secret, events array, URL; domain model/routes; campaign webhooks for email, SMS, and survey responses.	Important for connecting SurveyArc events to external systems and branded/custom domain workflows.	Needs webhook signing, retry policy, delivery logs, domain verification, and security documentation before production use.
Audit, archive, and metrics	Audit event routes by entity/org, audit log CRUD, audit services/helpers, archive model/routes, metric model/routes.	Useful for compliance, admin review, operational history, and internal reporting.	Needs consistent audit coverage across all important mutations and a clear retention policy.
Salesforce integration	Salesforce contacts/accounts/leads routes, account-with-contacts route, update/delete contact, sync contacts, add Salesforce contacts to list, sync accounts as lists, campaign preview/create-send/bulk-send flows, frontend Salesforce contacts/accounts pages and providers.	Strong additional value for B2B audience sourcing and survey campaign distribution from CRM data.	Should be treated as an integration module requiring credential setup, API limit handling, retry/error handling, and field mapping documentation.
Panel, anonymous link, QR, embed, participant sources	PanelManager, panel integration/overview components, AnonymousLinkPage, QrCodePage, EmbedPage, participant source APIs for generate URL, click/start/complete tracking, custom exit pages, and error messages.	Important for survey distribution outside email campaigns, including public links, embedded surveys, panel/source tracking, and respondent-source analytics.	Valuable for surveys, but needs final end-to-end validation with live respondent traffic.

Email and WhatsApp distribution surfaces	EmailPage, EmailListUploader, EmailTabsNav, email settings/provider cards, email templates editor/list/preview/variables, WhatsApp distribution page, WhatsApp config/contacts/lists/templates/template editor, WABA utilities.	Adds campaign/distribution UI beyond backend campaign APIs and mail relay.	Email is more concrete; WhatsApp requires provider/WABA credentials, template approval handling, and production send flow validation.
Analytics and exports	SurveyAnalyticsPanel, AnalyticsCard, response aggregation utilities, NPS analytics, checkbox aggregation, duplicate detection, graph helpers, Excel export, SPSS export, response formatting utilities.	Important for the post-survey value chain: not only collecting responses, but analyzing and exporting them.	Needs verification against real data and questionnaire edge cases before relying on it for client reporting.
Tags and taxonomy operations	Tag CRUD, counts, stats, categories, bulk create/delete, ticket tag usage recalculation, merge, cleanup unused, tag frontend page/form.	Useful for support/ticket organization and future project/survey categorization.	Mostly operational support value, with final QA required around usage counts and merge behavior.
Advanced support routing, capacity, and ML routing	Robust routing engine with policy/skills/capacity/SLA/geography/fallback layers, routing analytics, capacity overview, agent/team capacity, ML routing optimizer, optimization status, A/B test manager, routing health endpoints.	High optional value for the ticket/SLA side of the platform and a strong example of advanced internal operations thinking.	Should be sold as experimental/advanced foundation unless proven with live routing data.
Ownership transfer, settings, team administration	Accept ownership/ownership transfer API routes and components, organization settings, team invite/team components, policy/subscription setting components, group pages.	Supports internal administration and organization lifecycle beyond basic user CRUD.	Needs admin workflow documentation and role-based access review.
Security/auth/frontend utilities	Firebase admin core, auth routes, secure CRUD route, crypto client utilities, encryption helpers, request context middleware, locale-aware app routing, English/Hindi message files.	Shows work has been done on authenticated and internationalized platform behavior.	All secrets, service account files, private keys, and environment files should be removed from sale package or rotated before transfer.

Repo hygiene note: the working tree includes development/runtime artifacts such as `node_modules`, virtual environment folders, `.next`, `.env` files, key files, server certificates, and a Firebase service account credential. These should not be treated as product value. Before final handoff, the repository should be cleaned, secrets should be removed from history where practical, and all exposed credentials/keys should be rotated.

7. Current Technical Quality Assessment

The codebase is large and valuable, but it is not finished as a polished commercial product. The best framing is "substantial internal platform foundation requiring final engineering handoff and hardening." That framing is fair and helps justify a meaningful price without overclaiming.

Quality Area	Score	Reason
Feature breadth	9.2 / 10	Survey, project, users, contacts, campaigns, quota, translation, XML, ticketing, SLA, RBAC, mail relay, support, Salesforce, integrations, webhooks, pricing/payment foundations, audit/archive/metrics, analytics, exports, and distribution surfaces are all present.
Survey product foundation	8.4 / 10	Survey builder, question library, blocks, runtime, responses, quota, rules, translations, templates, and XML mode are materially developed.
Ticket/SLA foundation	7.5 / 10	Large support/ticketing/SLA system exists, but it should be treated as a secondary module needing QA.
Maintainability	6.5 / 10	Large files, route sprawl, repeated patterns, and incomplete cleanup create maintenance cost, but module boundaries are understandable.
Documentation and handoff readiness	7.5 / 10	Can be brought to 7-8 level if accompanied by the audit report, KT documentation, setup notes, module map, environment list, and known-issues list.
Automated quality coverage	3.5 / 10	No meaningful automated test suite was identified; regression risk remains.
Production hardening	5.8 / 10	Good internal foundation, but deployment, secrets, monitoring, migrations, provider integrations, and tests need work.
Internal-use acquisition suitability	8.0 / 10	For an internal team that can continue development, this is a strong code asset with meaningful time savings.

8. Development Effort Already Invested

Based on the product breadth, module count, and visible complexity, the stated development effort of approximately six months of continuous work at around five to six hours per day is consistent with the amount of code and feature surface present in the repository.

Effort Basis	Estimate	Comment
Calendar duration	Approximately 6 months continuous development	Consistent with a broad platform build rather than a small MVP.
Daily effort	Approximately 5 to 6 hours per day	Assumes sustained solo/founding engineer effort.
Total invested hours	Approximately 900 to 1,080 hours	Useful for explaining why the price may look high but is grounded in real effort.
Replacement effort from scratch	Approximately 1,100 to 1,600 engineering hours	A new team would spend additional time understanding requirements, rebuilding architecture, and rediscovering edge cases.
Stabilization effort after acquisition	Approximately 180 to 320 hours for survey-first handoff; 220 to 390 hours if every secondary module is production-hardened	Needed for tests, production hardening, bug cleanup, docs, deployment, final UX polish, integration validation, and credential cleanup.

9. Replacement Cost and Commercial Valuation

There are two different numbers to communicate: replacement cost and acquisition price. Replacement cost is what it would cost the buyer to build the same foundation from zero. Acquisition price is what is fair for transferring an incomplete but valuable existing code asset.

9.1 Replacement Cost Estimate

Rate Scenario	Hours Assumed	Cost Range	Use Case
Conservative independent/small-studio blended rate	1,100 to 1,600 hours	INR 6,60,000 to INR 14,40,000	Conservative labor-only estimate at roughly INR 600 to INR 900 per hour.
Professional agency/internal engineering replacement	1,100 to 1,600 hours	INR 13,20,000 to INR 32,00,000	More realistic if handled by a formal development team with PM, QA, backend, frontend, and DevOps overhead.
Commercial SaaS-level rebuild	1,800+ hours	INR 35,00,000+	Would include production QA, compliance, analytics, documentation, cloud hardening, and support tooling.

9.2 Recommended Acquisition Pricing

Offer Type	Recommended Price	Includes	Comment
Survey-focused code transfer only	INR 8,00,000 to INR 11,00,000	Survey builder, question library, runtime, responses, XML mode, quotas/rules, translations, themes, survey/project integration.	Use only if ticketing/SLA/support modules are excluded or treated as non-commercial bonus.
Full repository as-is transfer	INR 12,00,000 to INR 15,00,000	Survey product plus ticket/SLA/RBAC/user/project/contact/campaign/mail/key/Redis modules.	Best balanced ask for internal-use acquisition.
Full repository with KT and documentation bundle	INR 18,00,000 to INR 22,00,000	Full codebase, setup support, architecture walkthrough, module documentation, known issue list, environment guide, and 2-4 weeks of limited handoff support.	Recommended opening pitch because the buyer is asking for something their internal team can continue.
Strategic transfer plus extended support	INR 20,00,000 to INR 25,00,000	Everything above plus extended bug-fix support, deployment help, and first stabilization sprint.	Only pitch if they want you actively involved after handoff.

Recommended practical pitch: quote **INR 18,00,000** for the full repository with documentation and KT. Mention that you are open to discussion and can adjust the amount if they only want the survey portion or want reduced handoff support.

9.3 Individual Module Costing

The table below breaks the total value into individual module estimates. These are not separate mandatory sale prices; they are valuation components used to justify the overall ask.

Module	Estimated Standalone Value	Reasoning
Survey builder and survey management	INR 3,25,000 to INR 4,75,000	Survey CRUD, builder navigation, blocks, ordering, editor workflows, status, project integration.
Question type library and config UI	INR 2,50,000 to INR 4,00,000	Large set of basic, CX, matrix, pricing, research, media, and flow question types with dedicated configuration components.
Survey runtime and response collection	INR 1,50,000 to INR 2,50,000	Respondent-facing survey rendering, answer UI, response APIs, response screens, demo/preview support.
XML survey programming mode	INR 1,00,000 to INR 1,75,000	XML editor, serializer, compiler-style preview, save-to-survey flow, serial-label based references.
Rules, logic, quota	INR 1,50,000 to INR 2,50,000	Rule APIs, logic schema, quota APIs, quota models, quota frontend components, evaluate/increment lifecycle.
Translations and localization	INR 90,000 to INR 1,50,000	Translation structures, coverage, initialize/resync, CSV upload/export/validation, frontend translation workflow.
Themes, preview, templates	INR 80,000 to INR 1,25,000	Theme backend/frontend, survey preview/demo surfaces, template-based survey creation and bulk question creation.
Contacts, audience files, campaigns	INR 1,75,000 to INR 2,75,000	Contacts, lists, channels, audience upload, campaign CRUD/scheduling/results, participant source tracking.
Project, organization, user management	INR 1,75,000 to INR 2,75,000	Organizations, projects, users, members, statuses, milestones, tags, attachments, favorites, recent activity.
RBAC and permission management	INR 1,50,000 to INR 2,25,000	Roles, permissions, role mappings, custom grants, denials, effective permission logic, frontend RBAC screens.
Ticket management and SLA system	INR 2,75,000 to INR 4,50,000	Ticket CRUD, comments, attachments, collaborators, worklogs, assignment, SLA admin, support groups/teams/routing/templates.
Mail relay, key server, Redis, background services	INR 1,25,000 to INR 2,25,000	Dedicated mail relay, key server, encryption middleware, Redis service layer, scheduler/outbox processors.
Salesforce, panel, and external distribution foundations	INR 1,25,000 to INR 2,25,000	Salesforce contacts/accounts/sync/campaign routes, panel/source tracking, anonymous link, QR, embed, participant source tracking.
Commerce, integrations, webhooks, audit, exports	INR 1,50,000 to INR 2,75,000	Pricing/payment/order/invoice foundations, marketplace/integration/webhook/domain routes, audit/archive/metric models, analytics/export utilities.
Documentation and KT package	INR 75,000 to INR 1,25,000	Architecture notes, setup guide, environment guide, module walkthrough, known issue register, transition sessions.

9.4 Costing Summary

Valuation View	Amount	Comment
Sum of individual module value ranges	Approximately INR 24,00,000 to INR 39,00,000	This is a replacement/component value view, not the suggested as-is sale price.
Adjusted as-is internal-use acquisition value	INR 12,00,000 to INR 15,00,000	Accounts for incomplete testing, polish, documentation, and production hardening.
Opening quote with KT/documentation	INR 18,00,000 to INR 22,00,000	Good opening position because it includes knowledge transfer value.
Negotiation floor	INR 11,00,000 to INR 12,50,000	Below this, the effort and code asset value are not properly recognized.
Advance adjustment mentioned by buyer	INR 22,500	This can be adjusted from the final accepted amount if both parties agree.

10. What Is Still Left To Do

The remaining work should be presented clearly so the buyer understands the acquisition is an as-is foundation, not a fully packaged finished SaaS. This transparency makes the valuation more credible.

Priority	Workstream	Details	Estimated Effort
High	Survey QA and regression testing	Test create/edit/delete/order/save/preview/runtime for every question type, including templates and XML generated surveys.	45 to 75 hours
High	Survey save/status consistency	Finalize draft/live status source of truth, save button behavior, dirty-state indicator, and builder/list synchronization.	20 to 35 hours
High	XML compiler hardening	Define XML grammar, validation messages, duplicate serial checks, logic syntax, quota syntax, docs, and tests.	35 to 70 hours
High	RBAC cleanup and permission matrix	Finalize roles/permissions for survey-only buyer workflows, sidebar permission caching, admin access, and project scope behavior.	25 to 45 hours
Medium	Campaign/mail provider hardening	Finalize SMTP config, retry behavior, relay health monitoring, unsubscribe/tracking, and real SMS/WhatsApp provider integration if required.	30 to 60 hours
Medium	Quota and logic end-to-end flow	Connect quota/rules deeply into runtime survey flow with over-quota termination, respondent state, and reporting.	35 to 70 hours
Medium	Ticket/SLA stabilization	QA ticket creation, assignment, comments, SLA timers, pause/resume, notifications, support teams, routing, templates.	40 to 80 hours
Medium	Deployment and documentation	Environment variables, Docker runbook, service map, database/migration setup, seed data, known issues, KT videos/notes.	25 to 45 hours
Medium	Automated tests	Add backend unit/integration tests and frontend smoke/E2E tests for critical flows.	50 to 100 hours
Medium	Salesforce, payments, webhooks, audit, exports validation	Validate Salesforce credentials/mappings, payment verification, invoice/order state, webhook signing/retries, audit coverage, analytics/export correctness.	35 to 70 hours
High	Repository hygiene and credential rotation	Remove runtime/vendor/build artifacts from the transfer package, exclude environment files, rotate Firebase/service keys/certificates, and document required secrets.	10 to 20 hours
Low	Code cleanup	Remove generated dummy assets, reduce large-file complexity, clean duplicate route patterns, improve logs, standardize naming.	20 to 40 hours

11. Key Risks for Buyer

Risk	Severity	Explanation	Mitigation
Lack of automated tests	High	Large feature surface without formal tests creates regression risk during handoff.	Run a stabilization sprint focused on survey create/edit/runtime/campaign flows.
Large module complexity	Medium	Some files and routes are large, which makes onboarding slower.	Provide module map, KT sessions, and refactor only after buyer understands flows.
Provider integrations incomplete	Medium	SMS/WhatsApp/voice in mail relay are placeholders; email is more concrete.	Scope providers separately based on buyer needs.
Deployment/secrets hardening	Medium	Key server, encryption, relay, Redis, database, and external URLs need production configuration.	Create environment and deployment checklist before go-live.
Repository contains local artifacts and credentials	High	The repo tree includes development artifacts and sensitive-looking files such as env files, service-account credentials, keys, certificates, build folders, dependency folders, and virtual environments.	Clean the transfer package, rotate keys, move secrets to environment management, and document only safe placeholders.
Survey advanced question validation	Medium	Advanced question types are valuable but require edge-case QA.	Prioritize the question types the buyer actually needs for internal use.
RBAC/cache inconsistencies	Medium	Permission checks and Redis cache behavior need final cleanup to avoid repeated sidebar loads or stale permissions.	Document permission model and stabilize cache invalidation.

12. Recommended Sale Narrative

The recommended narrative is not "this is a finished SaaS." The stronger and more credible narrative is:

"This is a six-month, survey-first internal platform foundation. The survey module is already substantially built with a visual builder, many research question types, runtime flow, responses, translations, quota/rules, templates, XML programming mode, and project/user/campaign integration. Along with that, the repository includes additional operational modules such as ticketing, SLA, RBAC, user management, Salesforce, pricing/payment foundations, marketplace/integrations, webhooks, audit/archive/metrics, analytics/export utilities, mail relay, and Redis-backed services. It is being offered as-is for an internal engineering team to continue, with documentation and KT included."

12.1 Suggested Client Position

Point To Communicate	Suggested Wording
Why price is meaningful	The price may look high at first, but the repository represents around 900 to 1,080 hours of concentrated development and a replacement effort of 1,100 to 1,600 hours.
Why it is still fair	The ask is below a full replacement cost because the product still needs internal hardening, tests, final documentation, and buyer-specific customization.
Why survey value is central	The survey builder and question library alone represent substantial reusable value, especially with XML mode, quota, rules, translation, and campaign readiness.
Why ticketing should be mentioned	Ticketing/SLA/RBAC/user management are additional internal-platform assets included in the repository, even if the buyer initially focuses on surveys.
How to remain flexible	Offer full repository pricing first, then provide reduced scope options if they only want the survey module or less KT support.

13. Recommended Handoff Package

To make the transaction smoother, the code sale should include a structured handoff package. This makes the price easier to justify and reduces buyer anxiety.

Deliverable	Details
Repository transfer	Full cleaned source code including frontend, backend, key server, mail relay, Docker/config files, and current audit deliverables. Runtime/vendor/build artifacts and secrets should be excluded from the final transfer package.
Setup guide	Local development setup, services required, env variables, database/Redis/mail/key-server notes.
Architecture map	Survey, project, user/RBAC, contacts, campaigns, ticket/SLA, Salesforce, pricing/payment/order/invoice, marketplace/integrations/webhooks, audit/archive/metrics, mail relay, and key server module map.
Survey module guide	Question lifecycle, serial labels, internal IDs, blocks, question order, XML mode, templates, runtime flow, responses, quota/rules, translations.
Known issues list	Current bugs, warnings, incomplete integrations, generated files to ignore, credential rotation notes, and suggested first stabilization sprint.
KT sessions	2 to 4 sessions covering setup, survey builder, backend schema/API, RBAC/project/user management, and optional ticket/SLA module.
Post-handoff support	Optional limited support window for setup clarifications and first-run blockers.

14. Final Conclusion

SurveyArc should be valued primarily as a survey platform foundation with a substantial supporting internal operations layer. The survey portion alone contains enough built functionality to justify a meaningful acquisition price: visual builder, broad question library, XML programming mode, question/serial-label architecture, templates, survey runtime, responses, translations, quota, rules, themes, campaigns, and project/user integration.

The ticket management, SLA, RBAC, user management, organization/project management, contacts, campaigns, Salesforce, pricing/payment/order/invoice foundations, marketplace/integrations, webhooks/domains, audit/archive/metrics, analytics/export utilities, email/WhatsApp distribution surfaces, mail relay, Redis services, and key server should be presented as additional platform assets included in the transfer. They increase the buyer's optional future value even if their immediate use case is only internal survey management.

Recommended commercial position: open with **INR 18,00,000 to INR 22,00,000** for the full repository including documentation and KT. Expect realistic closure around **INR 14,00,000 to INR 17,00,000**. Avoid going below **INR 11,00,000 to INR 12,50,000** for full transfer because the codebase represents approximately six months of concentrated engineering work and a large replacement effort.